



University of Asia Pacific

DEPARTMENT OF CSE

COURSE CODE : 402

COURSE TITLE : Operating System Lab

DATE OF SUBMISSION : 17/08/2026

SUBMITTED BY:

NAME : Fairaj Salim

STUDENT ID : 23101165

SECTION : C2

LEVEL : 4th Year 1st Semester

SUBMITTED TO:

Atia Rahman Orthi

LECTURER

Department of CSE, UAP

PROBLEM STATEMENT

Implement and compare FCFS (First Come First Serve), SJF (Shortest Job First), and Round Robin CPU Scheduling algorithms for the following scenario where the number of processes = 6 and the process information is:

P_id	AT	BT
p1	0	7
p2	1	4
p3	2	15
p4	3	11
p5	4	20
p6	4	9

Calculate CT, TAT, WT, Average TAT and Average WT for each algorithm (Round Robin uses a Time Quantum = 5). Also represent the process execution sequence for each, and compare the three algorithms based on their performance.

SOURCE CODE

```
p_id = ["p1", "p2", "p3", "p4", "p5", "p6"]
at = [0, 1, 2, 3, 4, 4]
bt = [7, 4, 15, 11, 20, 9]

n = len(p_id)

fcfs_id = p_id.copy()
fcfs_at = at.copy()
fcfs_bt = bt.copy()

for i in range(n - 1):
    for j in range(n - 1 - i):
        if fcfs_at[j] > fcfs_at[j + 1]:
            fcfs_at[j], fcfs_at[j + 1] = fcfs_at[j + 1], fcfs_at[j]
            fcfs_bt[j], fcfs_bt[j + 1] = fcfs_bt[j + 1], fcfs_bt[j]
            fcfs_id[j], fcfs_id[j + 1] = fcfs_id[j + 1], fcfs_id[j]

fcfs_ct = []
fcfs_tat = []
fcfs_wt = []
time = 0
sum_tat = 0
sum_wt = 0

for i in range(n):
    if time < fcfs_at[i]:
```

```

        time = fcfs_at[i]
        time = time + fcfs_bt[i]
        fcfs_ct.append(time)
        fcfs_tat.append(fcfs_ct[i] - fcfs_at[i])
        fcfs_wt.append(fcfs_tat[i] - fcfs_bt[i])
        sum_tat = sum_tat + fcfs_tat[i]
        sum_wt = sum_wt + fcfs_wt[i]

fcfs_avg_tat = sum_tat / n
fcfs_avg_wt = sum_wt / n

sjf_ct = [0] * n
sjf_tat = [0] * n
sjf_wt = [0] * n
done = [0] * n
time = 0
completed = 0
sjf_seq = []

while completed < n:
    smallest = -1
    for i in range(n):
        if at[i] <= time and done[i] == 0:
            if smallest == -1 or bt[i] < bt[smallest]:
                smallest = i
    if smallest != -1:
        sjf_seq.append(p_id[smallest])
        time = time + bt[smallest]
        sjf_ct[smallest] = time
        sjf_tat[smallest] = sjf_ct[smallest] - at[smallest]
        sjf_wt[smallest] = sjf_tat[smallest] - bt[smallest]
        done[smallest] = 1
        completed = completed + 1
    else:
        time = time + 1

sjf_avg_tat = sum(sjf_tat) / n
sjf_avg_wt = sum(sjf_wt) / n

quantum = 5
rr_id = p_id.copy()
rr_at = at.copy()
rr_bt = bt.copy()

for i in range(n - 1):
    for j in range(n - 1 - i):
        if rr_at[j] > rr_at[j + 1]:
            rr_at[j], rr_at[j + 1] = rr_at[j + 1], rr_at[j]
            rr_bt[j], rr_bt[j + 1] = rr_bt[j + 1], rr_bt[j]
            rr_id[j], rr_id[j + 1] = rr_id[j + 1], rr_id[j]

remaining = rr_bt.copy()
rr_ct = [0] * n
rr_tat = [0] * n
rr_wt = [0] * n
queue = []
added = [0] * n
time = 0
completed = 0
rr_seq = []

```

```

while completed < n:
    for i in range(n):
        if rr_at[i] <= time and added[i] == 0:
            queue.append(i)
            added[i] = 1

    if len(queue) == 0:
        time = time + 1
    else:
        current = queue.pop(0)
        rr_seq.append(rr_id[current])

        if remaining[current] > quantum:
            time = time + quantum
            remaining[current] = remaining[current] - quantum
        else:
            time = time + remaining[current]
            remaining[current] = 0

        for i in range(n):
            if rr_at[i] <= time and added[i] == 0:
                queue.append(i)
                added[i] = 1

        if remaining[current] > 0:
            queue.append(current)
        else:
            rr_ct[current] = time
            rr_tat[current] = rr_ct[current] - rr_at[current]
            rr_wt[current] = rr_tat[current] - rr_bt[current]
            completed = completed + 1

rr_avg_tat = sum(rr_tat) / n
rr_avg_wt = sum(rr_wt) / n

print("\nFCFS:")
print("Execution Sequence:", end=" ")
for i in range(n):
    print(fcfs_id[i], end=" ")
print("\n")
print(f"{'PID':<8}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
for i in range(n):
    print(f"{'fcfs_id[i]':<8}{'fcfs_at[i]':<6}{'fcfs_bt[i]':<6}"
          f"{'fcfs_ct[i]':<6}{'fcfs_tat[i]':<6}{'fcfs_wt[i]':<6}")
print("\nAverage TAT =", fcfs_avg_tat)
print("Average WT  =", fcfs_avg_wt)

print("\nSJF:")
print("Execution Sequence:", end=" ")
for process in sjf_seq:
    print(process, end=" ")
print("\n")
print(f"{'PID':<8}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
for i in range(n):
    print(f"{'p_id[i]':<8}{'at[i]':<6}{'bt[i]':<6}"
          f"{'sjf_ct[i]':<6}{'sjf_tat[i]':<6}{'sjf_wt[i]':<6}")
print("\nAverage TAT =", sjf_avg_tat)
print("Average WT  =", sjf_avg_wt)

```

```

print("\nROUND ROBIN:")
print("Time Quantum =", quantum)
print("Execution Sequence:", end=" ")
for process in rr_seq:
    print(process, end=" ")
print("\n")
print(f"{'PID':<8}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
for i in range(n):
    print(f"{'rr_id[i]:<8}{'rr_at[i]:<6}{'rr_bt[i]:<6}'"
          f"{'rr_ct[i]:<6}{'rr_tat[i]:<6}{'rr_wt[i]:<6}'")
print("\nAverage TAT =", rr_avg_tat)
print("Average WT =", rr_avg_wt)

print("\nCOMPARISON:")
print(f"{'Algorithm':<15}{'Avg TAT':<12}{'Avg WT':<12}")
print(f"{'FCFS':<15}{fcfs_avg_tat:<12.1f}{fcfs_avg_wt:<12.1f}")
print(f"{'SJF':<15}{sjf_avg_tat:<12.1f}{sjf_avg_wt:<12.1f}")
print(f"{'Round Robin':<15}{rr_avg_tat:<12.1f}{rr_avg_wt:<12.1f}")

```

OUTPUT

```

C:\Users\lab\PycharmProjects\23101165_RR\.venv\Scripts\python.exe C:\Users

FCFS
Execution Sequence: p1 p2 p3 p4 p5 p6

PID    AT    BT    CT    TAT    WT
p1      0     7     7     7     0
p2      1     4    11    10     6
p3      2    15    26    24     9
p4      3    11    37    34    23
p5      4    20    57    53    33
p6      4     9    66    62    53

Average TAT = 31.666666666666668
Average WT  = 20.666666666666668

SJF
Execution Sequence: p1 p2 p6 p4 p3 p5

PID    AT    BT    CT    TAT    WT
p1      0     7     7     7     0
p2      1     4    11    10     6
p3      2    15    46    44    29
p4      3    11    31    28    17
p5      4    20    66    62    42

```

```
p6 4 9 20 16 7

Average TAT = 27.83333333333332
Average WT = 16.83333333333332

ROUND ROBIN
Time Quantum = 5
Execution Sequence: p1 p2 p3 p4 p5 p6 p1 p3 p4 p5 p6 p3 p4 p

PID  AT  BT  CT  TAT  WT
p1   0   7  31  31  24
p2   1   4   9   8   4
p3   2  15  55  53  38
p4   3  11  56  53  42
p5   4  20  66  62  42
p6   4   9  50  46  37

Average TAT = 42.166666666666664
Average WT = 31.166666666666668

COMPARISON:
Algorithm  Avg TAT  Avg WT
FCFS      31.7    20.7
SJF       27.8    16.8
Round Robin 42.2    31.2
```

DISCUSSION

In this experiment, three CPU scheduling algorithms — FCFS (First Come First Serve), SJF (Shortest Job First), and Round Robin — were implemented in Python and their performances were compared using Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

While extending the earlier FCFS and SJF program to include Round Robin, I faced several new challenges that I did not encounter before. The first problem was a module/syntax error while setting up the Round Robin logic — a few indentation and bracket mismatches caused the program to fail before it even reached the scheduling logic, so I had to go through the code line by line to locate and correct them.

The next major difficulty was handling the Time Quantum itself. Unlike FCFS and SJF, where a process runs to completion once selected, Round Robin required me to check at every step whether the remaining burst time of a process was greater than the quantum. If it was, the process had to be preempted and pushed back into the ready queue instead of being marked complete, and any process that arrived during that quantum also had to be added to the queue before the pre-empted process was re-queued. Getting

the order of these queue insertions correct took several attempts, since inserting them in the wrong order changed the entire execution sequence.

I also faced problems while printing the output correctly, since Round Robin produces a much longer execution sequence than FCFS or SJF (a process can appear multiple times as it gets pre-empted and resumed). The initial print statements were not formatted to handle repeated PIDs cleanly, so I had to adjust the loop that prints the execution sequence.

Comparing all three algorithms together in a single COMPARISON table was another challenge, because FCFS, SJF, and Round Robin each maintain their own separate lists for AT, BT, CT, TAT, and WT, and mixing them up while printing initially produced incorrect average values. I fixed this by keeping the variable names strictly separated with `fcfs_`, `sjf_`, and `rr_` prefixes and double-checking which list was being summed before computing each average.

Finally, I faced some issues with taking the input values themselves — since this problem set has six processes instead of five, and two of them (p5 and p6) share the same Arrival Time, the sorting and queue-insertion logic needed to correctly handle ties in Arrival Time as well, which was not required in the earlier FCFS/SJF-only version.

From the output, FCFS produced an execution sequence of $p1 \rightarrow p2 \rightarrow p3 \rightarrow p4 \rightarrow p5 \rightarrow p6$, with an average Turnaround Time of 31.7 and an average Waiting Time of 20.7. SJF produced the execution sequence $p1 \rightarrow p2 \rightarrow p6 \rightarrow p4 \rightarrow p3 \rightarrow p5$, with an average Turnaround Time of 27.8 and an average Waiting Time of 16.8. Round Robin, with a Time Quantum of 5, produced a much longer, pre-emptive execution sequence and resulted in an average Turnaround Time of 42.2 and an average Waiting Time of 31.2. Comparing the three, SJF gave the lowest average TAT and WT, followed by FCFS, while Round Robin performed the worst for this particular process set because of the high context-switching overhead caused by the long burst times relative to the small quantum.

Why is FCFS required in CPU Scheduling?

FCFS is required because it is one of the simplest and fairest CPU scheduling approaches. It executes processes in the order in which they arrive in the ready queue. It is easy to implement and understand, and it does not require complex calculations or continuous comparison of process characteristics. FCFS can be useful when simplicity, predictable ordering, and low scheduling overhead are more important than minimizing waiting time. However, it can suffer from the convoy effect, where a long process makes shorter processes wait for a long time.

When does SJF perform better than FCFS and Round Robin?

SJF generally performs better than FCFS and Round Robin when processes have different Burst Times, particularly when shorter processes are available while a longer process is waiting. By selecting the process with the shortest Burst Time first, SJF reduces the waiting time of shorter processes and often

results in a lower average Waiting Time and Turnaround Time. In this experiment, SJF performed the best of the three, with an average Waiting Time of 16.8 and an average Turnaround Time of 27.8, compared with 20.7/31.7 for FCFS and 31.2/42.2 for Round Robin.

Why did Round Robin perform worse here?

Round Robin is designed to give every process a fair share of CPU time through pre-emption, which makes it suitable for time-sharing systems and gives good response time. However, when the burst times are large compared to the quantum (as in this process set, where burst times range up to 20 while the quantum is only 5), processes get pre-empted repeatedly, which increases both the Turnaround Time and Waiting Time compared to non-pre-emptive algorithms like FCFS and SJF. This shows that Round Robin's performance is highly dependent on choosing an appropriate Time Quantum.

CONCLUSION

This experiment provided practical experience in implementing and comparing the FCFS, SJF, and Round Robin CPU scheduling algorithms using Python. During the implementation, I faced several new difficulties beyond the earlier FCFS/SJF-only version — including a module/syntax error while first setting up the Round Robin block, confusion while handling the Time Quantum and pre-emption logic, formatting problems while printing the longer Round Robin execution sequence, mix-ups while comparing all three algorithms' results together in one table, and handling input correctly for six processes with two processes sharing the same Arrival Time. I solved these issues by carefully separating variable names with algorithm-specific prefixes, tracing the queue-insertion order step by step, and testing the program incrementally after each fix.

The comparison of the results shows that SJF performed the best among the three algorithms for the given set of processes, with an average TAT of 27.8 and an average WT of 16.8. FCFS followed with an average TAT of 31.7 and an average WT of 20.7, while Round Robin performed the worst, with an average TAT of 42.2 and an average WT of 31.2, mainly due to the high pre-emption overhead caused by the small Time Quantum relative to the burst times.

Overall, FCFS remains useful because of its simplicity, fairness, and ease of implementation; SJF is the most efficient in terms of minimizing average waiting and turnaround time but requires knowledge of burst times in advance and may starve longer processes; and Round Robin ensures fairness and good response time in time-sharing systems, though its performance depends heavily on selecting an appropriate Time Quantum. This experiment helped me understand the practical differences between FCFS, SJF, and Round Robin, and further improved my programming, debugging, and problem-solving skills.